

# PHP OOP — "Student"

## Example

- PHP supports **OOP (Object-Oriented Programming)**
- OOP makes it easier to handle **complex data**
  - **Class** = blueprint
  - **Object** = thing built from the blueprint
  - Start **simple**, add features later

# 1. Define a Student Class

Three main components:

1. Fields (variables in a class)
2. Constructor
3. Methods (functions in a class)

# Fields

- The Student class has fields (variables in an object).
- `public` means any other object can access it.

```
<?php
declare(strict_types=1);

/**
 * A Student class:
 */
class Student {
    public int $id;
    public string $name;
    public string $email;
```

# Constructor

- A constructor (`__construct`) is called when this class is instantiated (created) as an object.
- `$this` means this object, and we use the `->` operator to access any element in the object.

```
public function __construct(  
    int $id, string $name, string $email) {  
    $this->id = $id;  
    $this->name = $name;  
    $this->email = $email;  
}
```

# Methods

- A function in a class is called `method` .
- `$this` means this object, so `$this->name` means the name field in the object.

```
public function greet(): string {  
    return "Hi, I'm {$this->name}!";  
}
```

## 2. Create and Use a Student Object

```
<?php
require 'Student.php'; // if you split files; else ignore

$s1 = new Student(1, 'Alice Johnson', 'alice@university.edu');

echo $s1->greet();           // "Hi, I'm Alice Johnson!"
echo PHP_EOL;
echo $s1->email;              // direct access (simple start)
```

- new Student(...) builds an object; the constructor is called automatically.
- Methods like greet() are the object's behavior.
- We use the `->` operator to access the method of an object.

### 3. Convert a Student Object to an Array

- We use the toArray() method (a function in a class) to return an array.

```
// Continued inside the Student class
/** Convert to an array */
public function toArray(): array {
    return [
        'id'      => $this->id,
        'name'    => $this->name,
        'email'   => $this->email,
    ];
}
```



- `toArray()` converts the PHP `Student` object into an array.
- `json_encode()` converts that array into a JSON string for an HTTP response.
- The conversion is: PHP object → array → JSON string.

```
$s1 = new Student(1, 'Alice Johnson', 'alice@university.edu');  
echo json_encode($s1->toArray(), JSON_PRETTY_PRINT);  
/*  
{  
  "id": 1,  
  "name": "Alice Johnson",  
  "email": "alice@university.edu"  
}  
*/
```

## IMPORTANT

Converting PHP objects to JSON strings, and JSON strings back to PHP data, is important.

- Server-side code works with PHP objects.
- Servers send JSON strings to clients such as web browsers.
- This conversion lets both sides exchange structured data.

## 4. Run student\_test.php

In the VS Code Terminal, start the server from the folder containing `student_test.php` :

```
php -S localhost:8000
```

1. You can choose any available port number.

2. Open

```
http://localhost:8000/student_test.php
```

in a web browser.

3. Use `Ctrl-C` to stop the server.

## Run with the PHP Server Extension

Alternatively, use the `PHP Server` extension in VS Code.

1. Open the `student_test.php`
2. Click the PHP icon .
3. Web browser will be opened with `http://localhost:3000/student_test.php`.
4. Use `Cmd-Shift-P` (Mac) or `Ctrl-Shift-P` (PC) to open command.
5. Choose "PHP Server:Stop server" to stop the server.

